**UNIVERSIDADE DO OESTE DE SANTA CATARINA - UNOESC**

Práticas Extensionistas IV

# JCondo

Sistema de Gestão de Condomínio

Jorge Luiz do Nascimento

Chapecó – SC 27 de Setembro de 2026

# 1. Integrantes do grupo

| # | Nome |
|---|---|
| 1 | Jorge Luiz do Nascimento |

# 2. Repositório no GitHub

> https://github.com/jorgensc/PEIV

O repositório segue o modelo da disciplina, com o README (título, autor, introdução e palavras-chave), a pasta `app` com o código e a pasta `doc` com este PDF e os três diagramas.

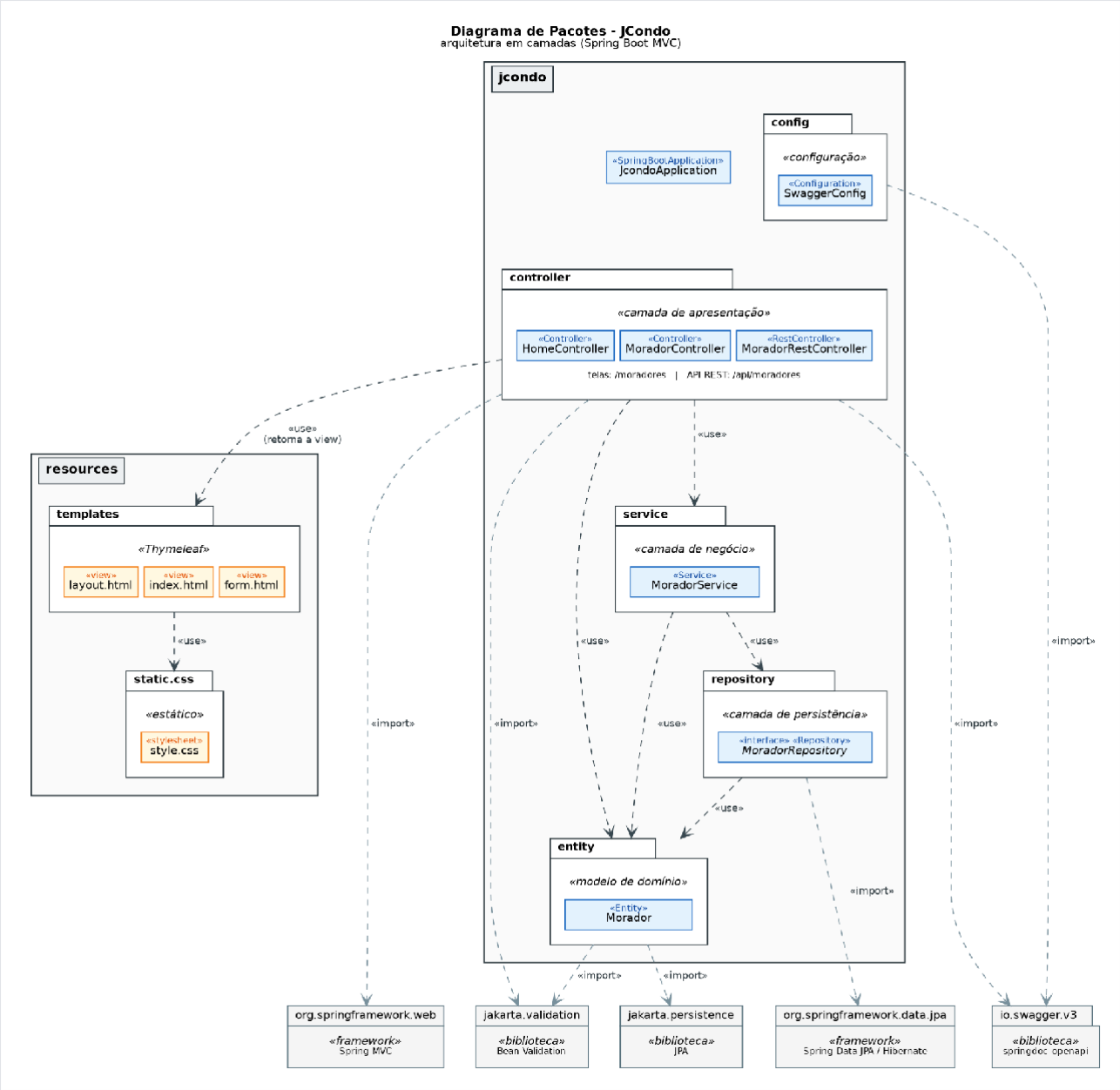
| Caminho | Conteúdo |
|---|---|
| README.md | título, autor, introdução, palavras-chave e os diagramas |
| app/ | aplicação Spring Boot (código, testes, script do banco e manuais em app/docs) |
| doc/ | este PDF e os diagramas de pacotes, de implantação e DevOps |
| .github/workflows/ci-cd.yml | pipeline de CI/CD no GitHub Actions com deploy no Azure |

## Sobre a aplicação

O JCondo é um sistema web de gestão de condomínio. Nesta versão ele faz o cadastro completo de moradores (criar, listar, editar e excluir) pelas telas web e por uma API REST documentada com Swagger. A stack usada é Java 21, Spring Boot 3.5 (Spring MVC, Spring Data JPA e Bean Validation), Thymeleaf com Bootstrap 5, MySQL 8, Maven e testes com JUnit 5 e Mockito.

Na entrega da PE III a stack prevista era PHP com XAMPP. No desenvolvimento o backend foi feito em Java com Spring Boot, mantendo o MySQL. Os diagramas das seções 3, 4 e 5 representam o código que está no repositório. Os diagramas de modelagem da PE III (caso de uso, classes, sequência e DER) continuam valendo e estão no anexo, no final deste documento.

# 3. Diagrama UML de pacotes (arquitetura da aplicação)



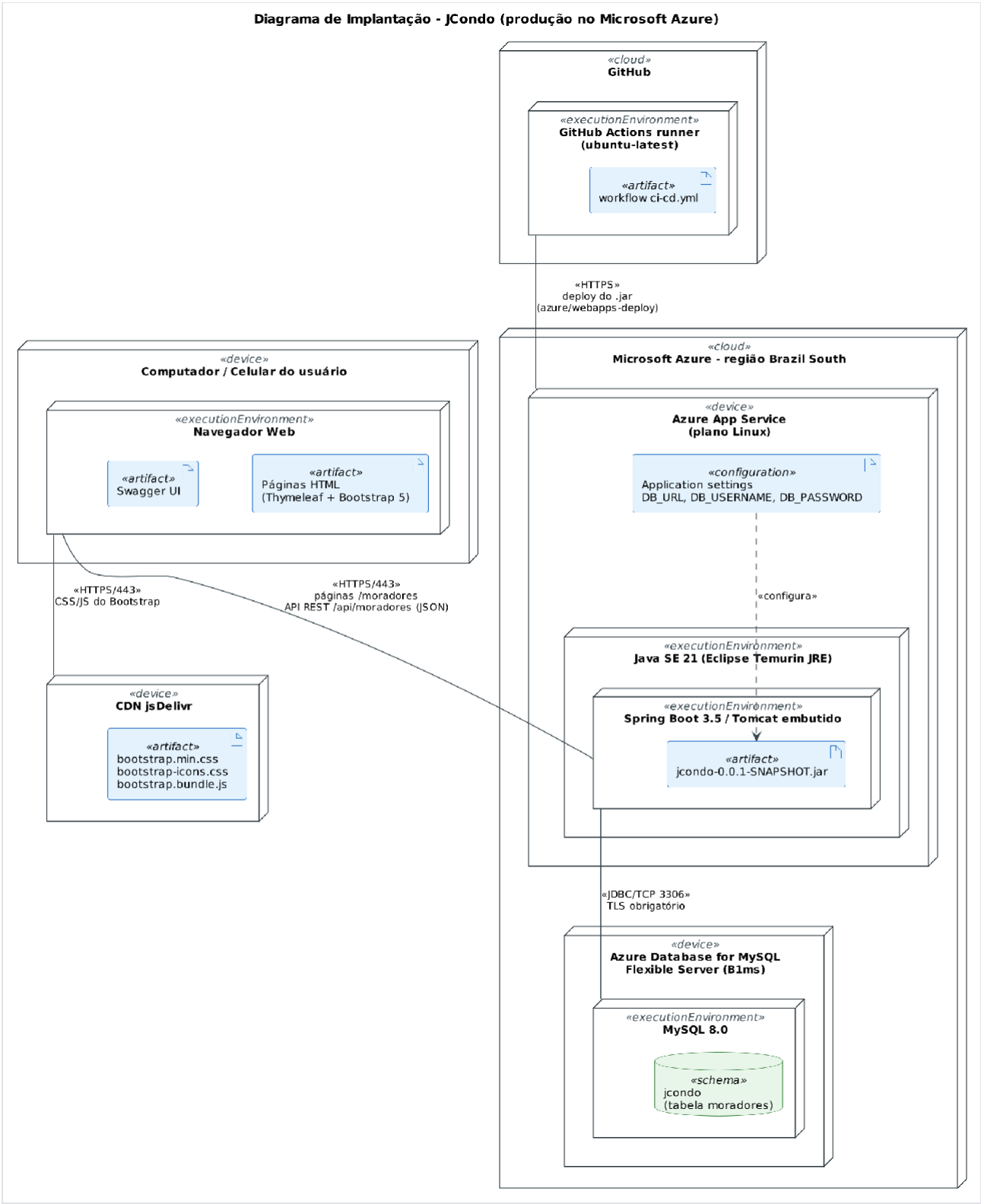
## Descrição do diagrama de pacotes

A aplicação segue arquitetura em camadas. Cada pasta com aba é um pacote, os retângulos internos são as classes, e as setas tracejadas são dependências: `«use»` para dependências entre pacotes do próprio sistema e `«import»` para as bibliotecas externas declaradas no `pom.xml`.

| Pacote | Responsabilidade |
|---|---|
| `jcondo` | pacote raiz, contém a classe `JcondoApplication` que sobe o Spring Boot |
| `controller` | camada de apresentação. `MoradorController` atende as telas (`/moradores`), `MoradorRestController` atende a API REST (`/api/moradores`) e `HomeController` redireciona a raiz |
| `service` | camada de negócio. `MoradorService` concentra as regras e as transações |
| `repository` | camada de persistência. `MoradorRepository` é uma interface Spring Data JPA |
| `entity` | modelo de domínio. `Morador` é a entidade JPA com as validações de cada campo |
| `config` | `SwaggerConfig`, configuração da documentação OpenAPI |
| `templates` / `static.css` | telas Thymeleaf (`layout`, `index` e `form`) e folha de estilo |

A regra da arquitetura é que as dependências só descem uma camada: o controller usa o service, o service usa o repository, e todos conhecem a entity. O controller nunca acessa o repository direto, o que deixa a regra de negócio isolada e testável sem banco (os testes do `MoradorService` usam o repository mockado com Mockito).

# 4. Diagrama de arquitetura de implantação



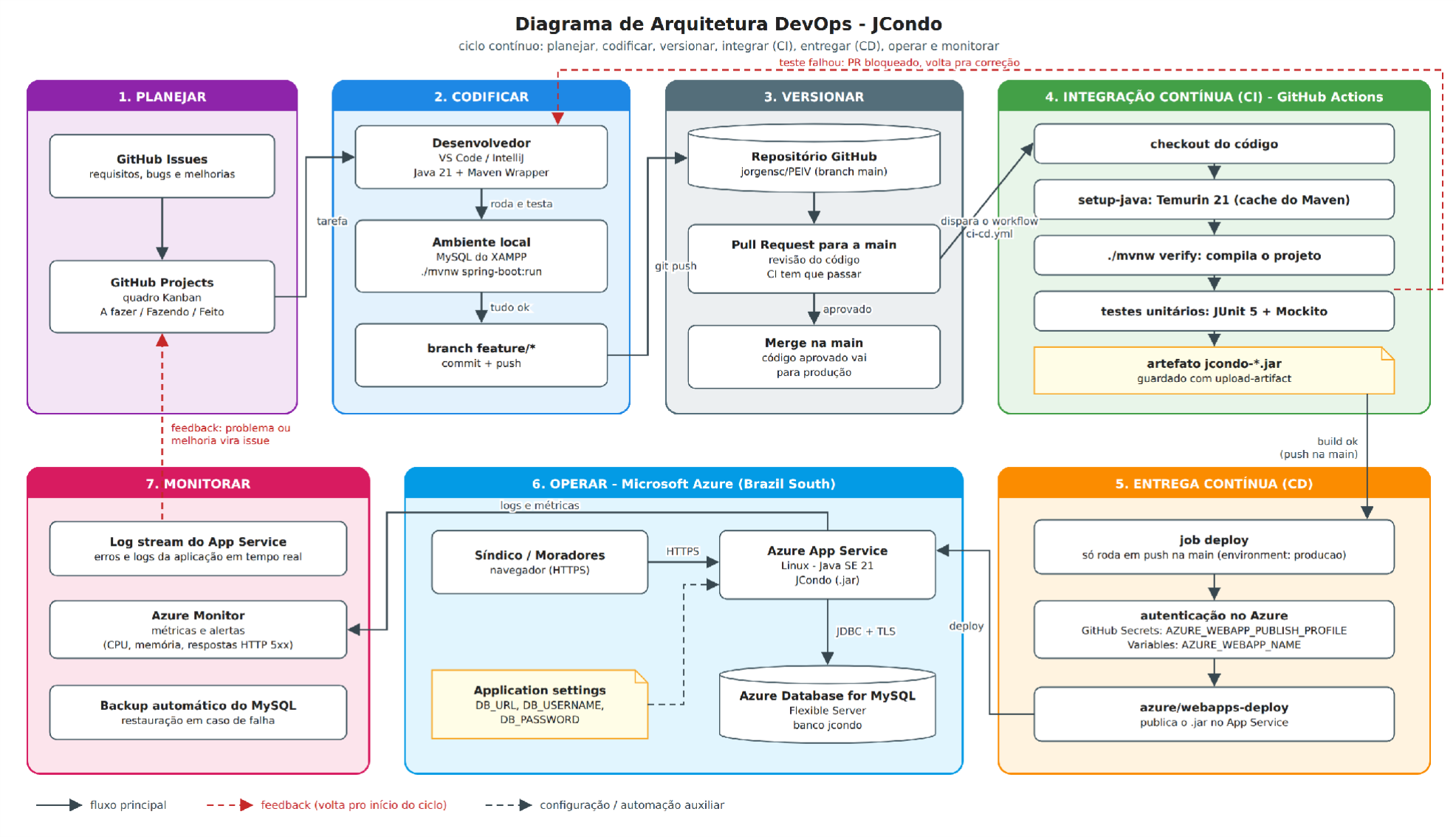
## Descrição do diagrama de implantação

O diagrama usa a notação UML de implantação: nós (`«device»` para hardware/infraestrutura e `«executionEnvironment»` para o software que hospeda outros artefatos), artefatos implantados e caminhos de comunicação com o protocolo.

| Nó | O que roda |
|---|---|
| Computador / celular do usuário | navegador web exibindo as páginas HTML geradas pelo Thymeleaf e o Swagger UI |
| CDN jsDelivr | arquivos do Bootstrap e Bootstrap Icons usados pelas telas |
| Azure App Service (Linux) | JRE Java 21 com o Spring Boot / Tomcat embutido executando o artefato `jcondo-0.0.1-SNAPSHOT.jar`. As credenciais do banco ficam nas Application settings, lidas como variáveis de ambiente |
| Azure Database for MySQL - Flexible Server | MySQL 8.0 com o banco `jcondo` (tabela `moradores`) |
| GitHub Actions runner | executa o workflow `ci-cd.yml` e envia o `.jar` para o App Service |

Comunicação: navegador e App Service por HTTPS na porta 443 (telas e API REST em JSON), navegador e CDN por HTTPS, App Service e MySQL por JDBC sobre TCP 3306 com TLS obrigatório, e GitHub Actions para o App Service por HTTPS no momento do deploy.

# 5. Diagrama de arquitetura DevOps



## Etapas do ciclo DevOps

1. **Planejar:** requisitos, bugs e melhorias são registrados como issues no GitHub e organizados num quadro Kanban no GitHub Projects.
2. **Codificar:** desenvolvimento com Java 21 e Maven Wrapper. Localmente a aplicação roda com `./mvnw spring-boot:run` usando o MySQL do XAMPP. Cada tarefa é feita numa branch `feature/*`.
3. **Versionar:** o código vai para o repositório no GitHub e entra na `main` por Pull Request, que só pode ser aprovado com o CI passando. Depois de aprovado, o merge na `main` libera o código para produção.
4. **Integração contínua (CI):** a cada push ou PR o GitHub Actions baixa o código, instala o JDK 21 com cache do Maven, roda `./mvnw verify` (compila e executa os testes JUnit 5 + Mockito) e guarda o `.jar` como artefato. Se um teste falhar, o PR fica bloqueado e volta para correção.
5. **Entrega contínua (CD):** em push na `main` o job de deploy roda no environment `producao`, autentica no Azure com o publish profile guardado nos GitHub Secrets e publica o `.jar` no App Service com a action `azure/webapps-deploy`.
6. **Operar:** o App Service executa a aplicação com HTTPS, lendo a conexão com o banco das Application settings, e o MySQL Flexible Server guarda os dados.
7. **Monitorar:** logs em tempo real pelo Log stream, métricas e alertas no Azure Monitor e backup automático do MySQL. Problemas e melhorias encontrados aqui viram novas issues, fechando o ciclo.

| Arquivo no repositório | Função no fluxo |
|---|---|
| `.github/workflows/ci-cd.yml` | pipeline de CI (build e testes) e CD (deploy no Azure) |
| `app/src/main/resources/application.properties` | lê `DB_URL`, `DB_USERNAME` e `DB_PASSWORD` do ambiente, com valores padrão para o XAMPP |

# 6. Infraestrutura de deploy / publicação

## 6.1 Escolha

A infraestrutura escolhida é a nuvem **Microsoft Azure**, pela assinatura **Azure for Students**, usando dois serviços gerenciados (PaaS):

| Serviço | Configuração | Uso |
|---|---|---|
| Azure App Service | plano Linux, runtime Java SE 21 (servidor web embutido). Plano F1 gratuito na fase acadêmica, com opção de subir para o B1 se precisar de mais desempenho | executar o `.jar` do Spring Boot |
| Azure Database for MySQL - Flexible Server | MySQL 8.0, camada Burstable B1ms | banco `jcondo` |

Região sugerida: Brazil South (São Paulo), pela menor latência para os usuários. Se a assinatura de estudante não liberar essa região para algum dos serviços, a alternativa é East US.

## 6.2 Justificativa

- **Custo zero para estudante e sem cartão de crédito.** A Azure for Students dá US$ 100 de crédito para usar em 12 meses, renovável a cada ano enquanto a matrícula estiver ativa, e o cadastro é feito com o e-mail da universidade. Além do crédito, a assinatura inclui serviços gratuitos que cobrem o que o projeto usa, como o App Service e 750 horas do MySQL Flexible Server.
- **Encaixa direto na stack do projeto.** O App Service tem runtime Java 21 pronto e roda o `.jar` do Spring Boot sem configurar servidor. O Azure Database for MySQL é o mesmo MySQL usado localmente, então o código não muda, só as variáveis de ambiente da conexão.
- **PaaS em vez de máquina virtual.** Não é preciso administrar sistema operacional, atualizações de segurança, firewall ou certificado. O App Service já entrega HTTPS no domínio `*.azurewebsites.net` e o MySQL gerenciado faz backup automático.
- **Integração com o GitHub.** O deploy usa a action oficial `azure/webapps-deploy`, então o mesmo pipeline que roda os testes publica a aplicação.
- **Espaço para crescer.** Basta trocar o plano do App Service (F1, B1, S1) ou a camada do banco, sem mudar a arquitetura.

## 6.3 Comparação com as outras opções

| Opção | Pontos a favor | Por que não foi a escolhida |
|---|---|---|
| **Azure for Students** (escolhida) | crédito anual sem cartão, PaaS para Java 21 e MySQL gerenciado, integração com GitHub Actions | tem limite de crédito e de serviços da conta de estudante, mas é suficiente para o porte do projeto |

| | | |
|---|---|---|
| AWS (Elastic Beanstalk + RDS) | maior provedor do mercado, RDS MySQL maduro | o plano gratuito atual dá até US$ 200 em créditos por no máximo 6 meses e depois a conta é encerrada. A configuração (VPC, security groups, IAM) é mais trabalhosa para o porte do projeto |
| Google Cloud (Cloud Run + Cloud SQL) | Cloud Run é simples e escala a zero | crédito de teste temporário, exige cartão de crédito e o Cloud SQL consome o crédito rápido |
| Railway / Render | deploy muito fácil a partir do GitHub | o Render não oferece MySQL gerenciado (só PostgreSQL) e o Railway passa a ser pago depois do crédito de teste |
| Oracle Cloud Always Free (VM + Docker) | máquina virtual gratuita sem prazo | pede cartão no cadastro e é IaaS: sistema operacional, Docker, firewall, certificado e backup ficam por minha conta |
| Self-host (servidor próprio) | nenhum custo de nuvem e controle total | depende de máquina ligada 24 horas, IP público ou túnel, energia e internet estáveis, e toda a segurança fica por minha conta, sem garantia de disponibilidade |

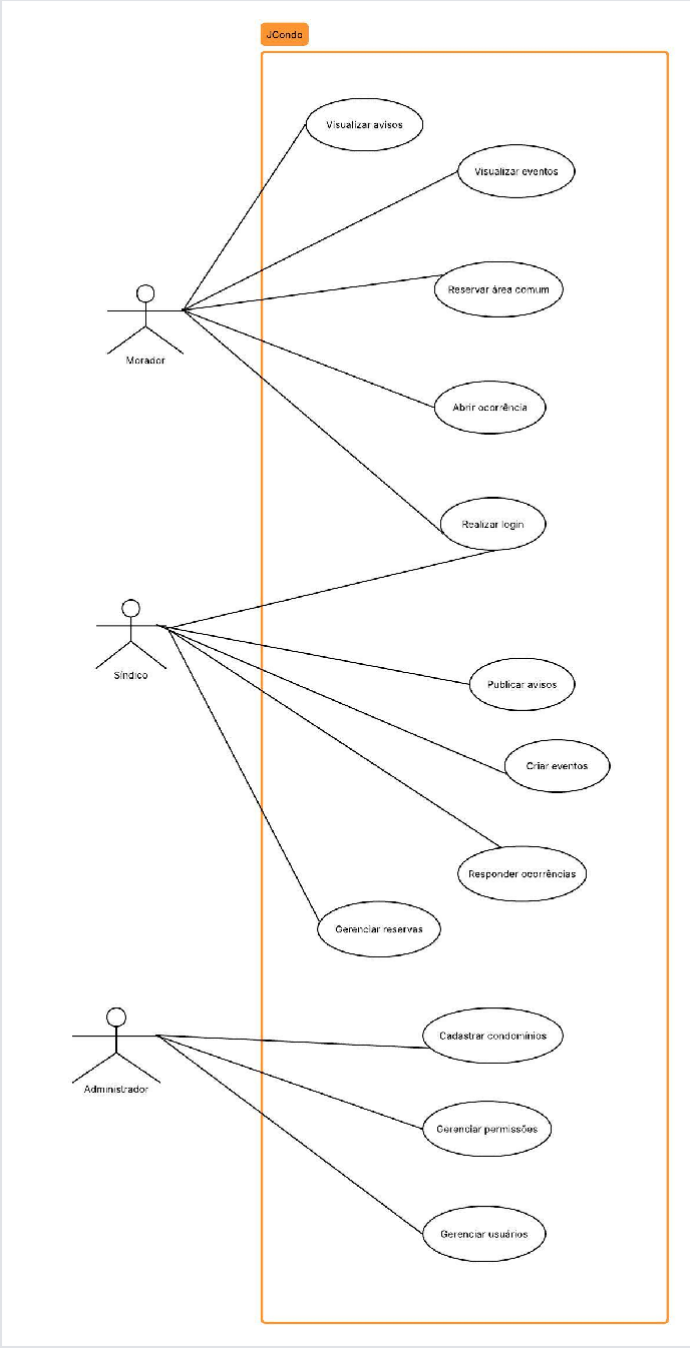
## 6.4 Ativação da conta e da estrutura

1. Ativar a assinatura em `azure.microsoft.com/free/students` com o e-mail institucional da UNOESC.
2. No portal do Azure, criar o grupo de recursos `rg-jcondo` na região Brazil South.
3. Criar o **Azure Database for MySQL - Flexible Server** (MySQL 8.0, Burstable B1ms), definir o usuário administrador e liberar o acesso de serviços do Azure na rede. Criar o banco `jcondo` e importar o script `app/database/jcondo.sql`.
4. Criar o **App Service** (publicação por código, pilha Java 21, Java SE, Linux) no plano F1.
5. Nas variáveis de ambiente do App Service, cadastrar `DB_URL` (`jdbc:mysql://<servidor>.mysql.database.azure.com:3306/jcondo?sslMode=REQUIRED`), `DB_USERNAME` `DB_PASSWORD` e `SHOW_SQL=false`.
6. Habilitar as credenciais de publicação básica (SCM) do App Service e baixar o perfil de publicação.
7. No GitHub, em *Settings* > *Secrets and variables* > *Actions*, cadastrar o secret `AZURE_WEBAPP_PUBLISH_PROFILE` e a variable `AZURE_WEBAPP_NAME`.
8. Fazer push na `main` ou executar o workflow manualmente. O pipeline compila, testa e publica, e o sistema fica disponível em `https://<nome-do-app>.azurewebsites.net`.

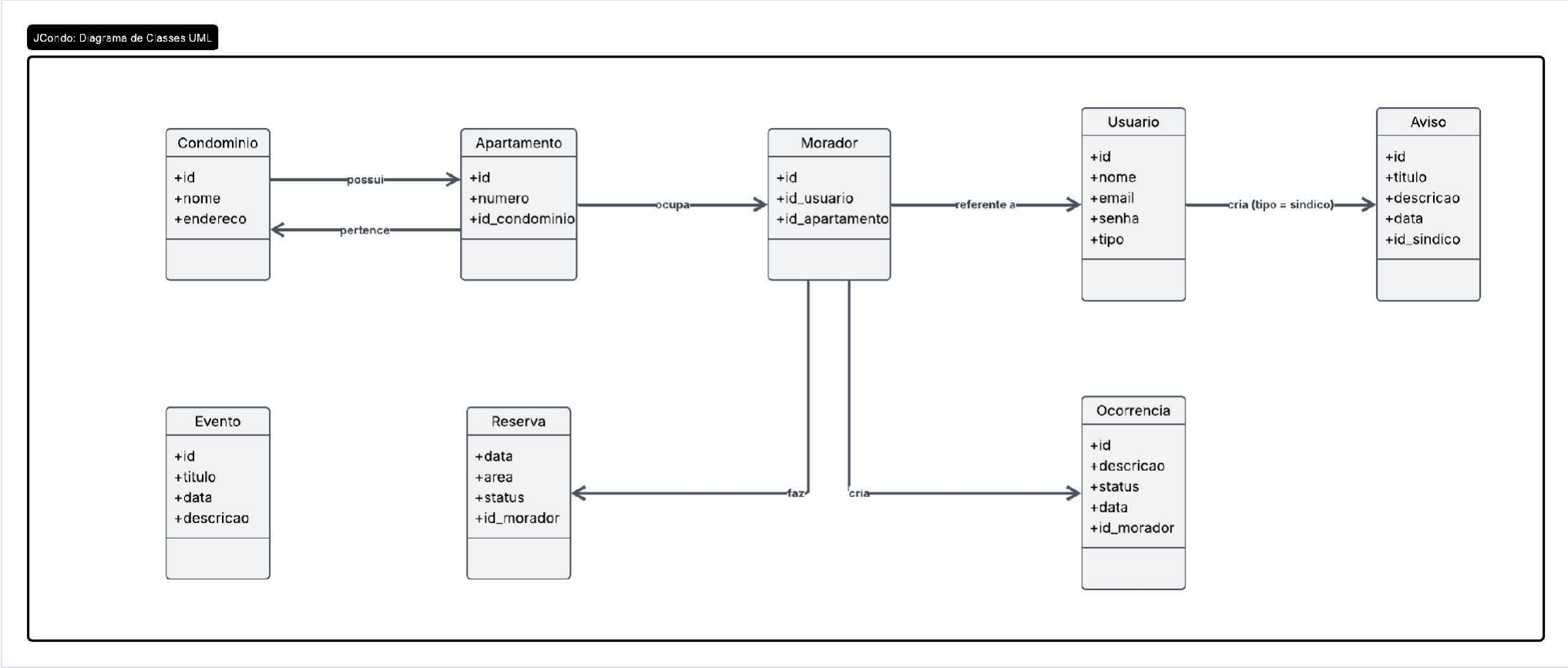
# Modelagem do sistema

Os diagramas abaixo representam o sistema completo planejado (moradores, avisos, reservas, eventos e ocorrências). O cadastro de moradores é o módulo já implementado no repositório.

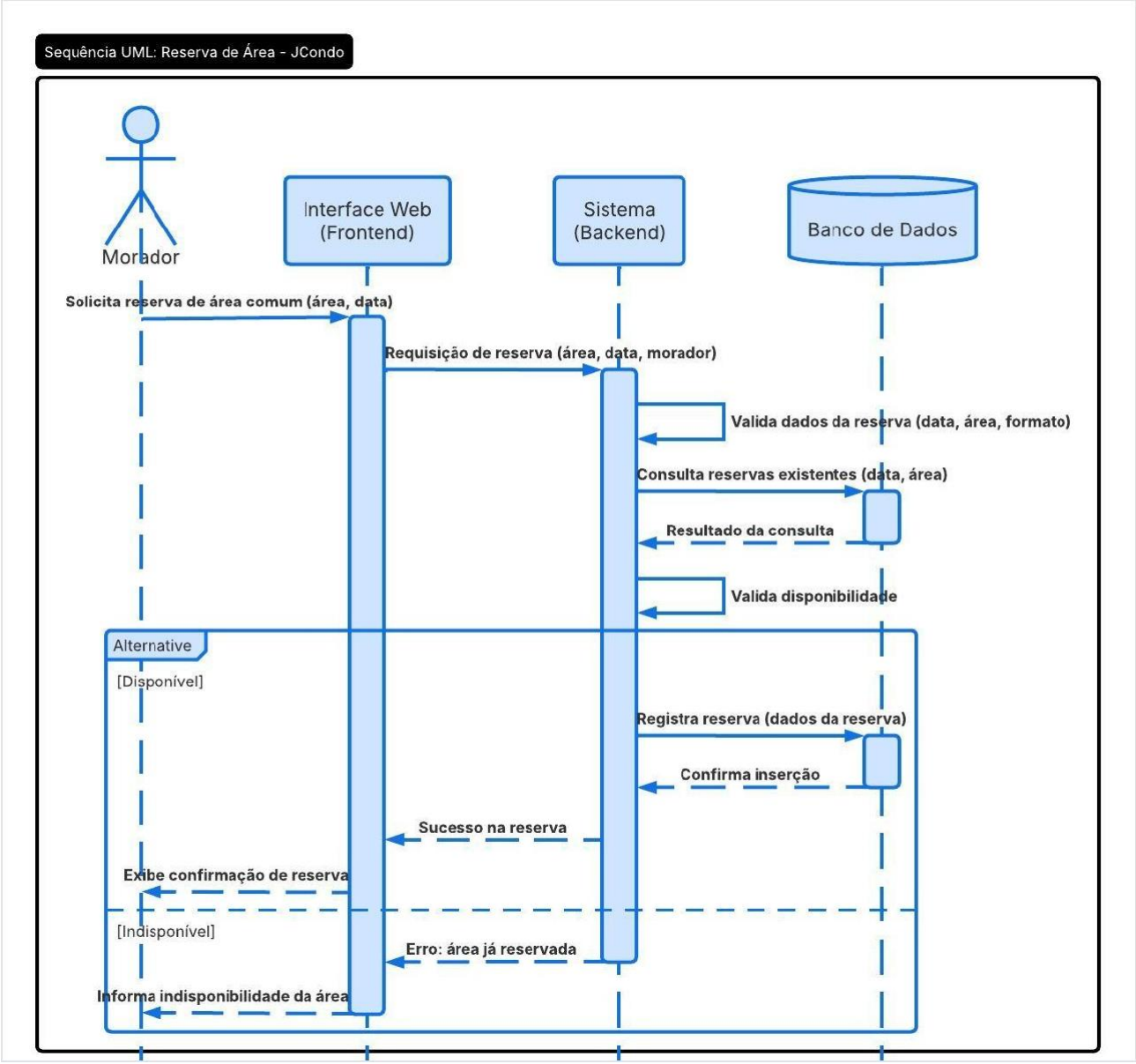
## A.1 Diagrama de caso de uso



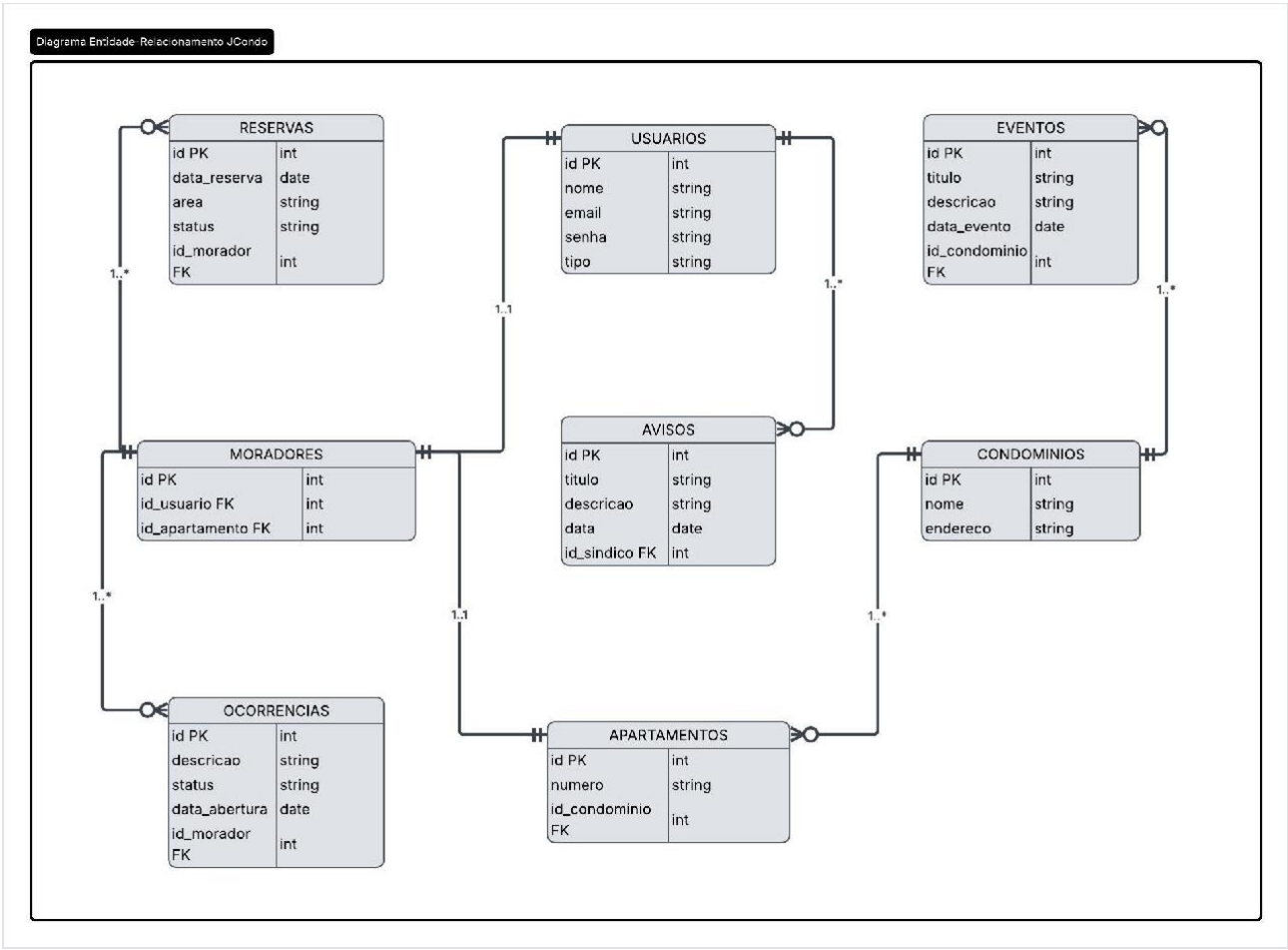
## A.2 Diagrama de classes



## A.3 Diagrama de sequência (reserva de área comum)



## A.4 Diagrama entidade-relacionamento (DER)




Entidades: usuarios, condominios, apartamentos, moradores, avisos, reservas, eventos e ocorrências. Um condomínio possui vários apartamentos, cada apartamento pertence a um condomínio, o morador é vinculado a um apartamento e pode fazer reservas e abrir ocorrências, e o síndico publica os avisos.